01. Análisis de Precipitaciones y Demanda de Subte

Autor: Grupo 36 **Fecha:** 2026

Contents

Incidencia de las Condiciones Climáticas en la Demanda de Pasajeros del Subte (2018–2021) **1**
Cátedra de Metodología de la Investigación — Licenciatura en Ciencia de Datos (UGR) 1
Grupo 36: Marco Fouad Abboud, Matías Gabriel Gallardo, Néstor Fabián León, Sebastián Luna 1
Propósito del Notebook: 1
1. Carga del Dataset Climático Consolidado 1
2. Análisis Exploratorio de Precipitaciones en CABA 2
3. Integración con Demanda de Subtes 2
4. Contraste de Hipótesis: Precipitaciones y Demanda 3
5. Modelo Econométrico Multivariado (OLS) y Diagnóstico de Supuestos 3

Incidencia de las Condiciones Climáticas en la Demanda de Pasajeros del Subte (2018–2021)

Cátedra de Metodología de la Investigación — Licenciatura en Ciencia de Datos (UGR)

Grupo 36: Marco Fouad Abboud, Matías Gabriel Gallardo, Néstor Fabián León, Sebastián Luna

---

Propósito del Notebook:

1. **Auditoría y Exploración (EDA):** Analizar el nuevo dataset de **precipitaciones diarias** y variables térmicas para CABA (2018–2021).
2. **Justificación Metodológica:** Reemplazar la métrica mensual previa por series diarias de alta resolución (Copernicus ERA5 y NASA POWER).
3. **Contraste Empírico de Hipótesis:**
   - Sensibilidad térmica diaria ($H_1A : \beta_{\text{temp}} < 0$).
   - Impacto de precipitaciones moderadas/significativas ($H_1B : d \geq 0.50$ en días > 5 mm).
4. **Modelado Econométrico Multivariado (OLS):** Diagnóstico de supuestos de Gauss-Markov (VIF, Durbin-Watson, Breusch-Pagan).

```
import os
from pathlib import Path
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from scipy import stats
from statsmodels.regression.linear_model import OLS
from statsmodels.tools.tools import add_constant
from statsmodels.stats.outliers_influence import variance_inflation_factor
from statsmodels.stats.diagnostic import het_breuschpagan
from statsmodels.stats.stattools import durbin_watson

# Configuracion visual
plt.style.use('seaborn-v0_8-whitegrid')
plt.rcParams['figure.figsize'] = (10, 5)
plt.rcParams['font.size'] = 10
print('Entorno y modulos econometricos inicializados con exito.')
```

```
Entorno y modulos econometricos inicializados con exito.
```

1. Carga del Dataset Climático Consolidado

El dataset fue generado a partir de las fuentes globales **ECMWF ERA5 (Copernicus C3S)** y **NASA POWER**, integrando además la estación sinóptica **SABA (Aeroparque Jorge Newbery)** vía Meteostat.

```
# Carga de datos climaticos diarios (2018-2021)
path_clima = Path('clima_diario_caba_2018_2021_consolidado.csv')
if not path_clima.exists():
    path_clima = Path('../datos/clima_diario_caba_2018_2021_consolidado.csv')

df_clima = pd.read_csv(path_clima, parse_dates=['fecha'])
print('Dimensiones dataset clima:', df_clima.shape)
df_clima[['fecha', 'precipitacion_mm', 'temperatura_media_c', 'es_dia_habil', 'es_covid_aspo']].head()
```

```
Dimensiones dataset clima: (1461, 32)
```

```
       fecha  precipitacion_mm  temperatura_media_c  es_dia_habil  \
0 2018-01-01               0.0                 22.1             1
1 2018-01-02               0.6                 25.3             1
2 2018-01-03               0.0                 25.8             1
3 2018-01-04               0.6                 28.8             1
4 2018-01-05               4.4                 22.8             1

   es_covid_aspo
0              0
1              0
2              0
3              0
4              0
```

2. Análisis Exploratorio de Precipitaciones en CABA

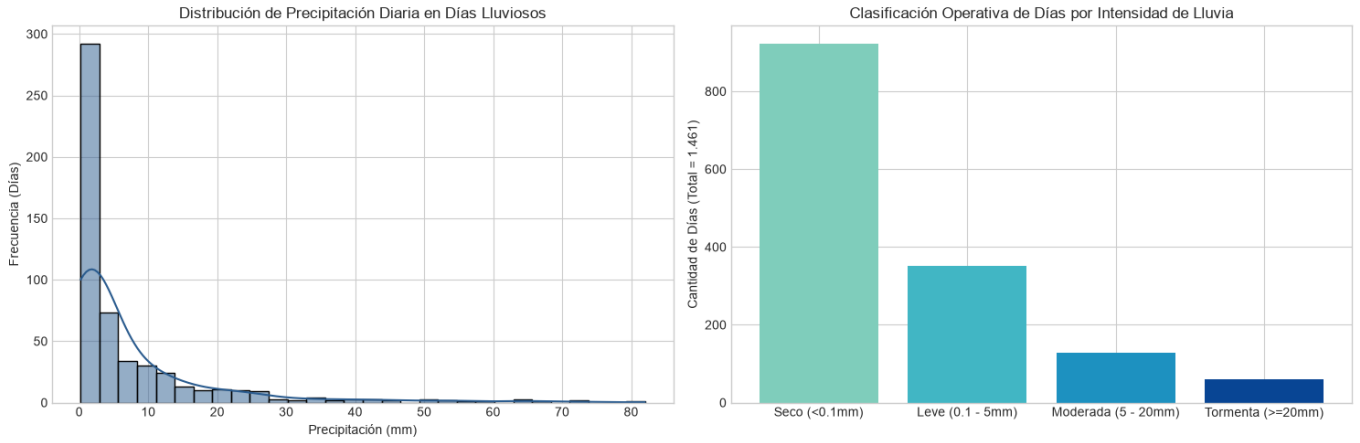
Examinamos la distribución de lluvia diaria y los umbrales de lluvia moderada (≥ 5 mm) y tormenta fuerte (≥ 20 mm).

```
fig, axes = plt.subplots(1, 2, figsize=(15, 5))

# Histograma de precipitaciones en días con lluvia
dias_con_lluvia = df_clima[df_clima['precipitacion_mm'] > 0.1]['precipitacion_mm']
sns.histplot(dias_con_lluvia, bins=30, kde=True, ax=axes[0], color='#2b5c8f')
axes[0].set_title('Distribución de Precipitación Diaria en Días Lluviosos')
axes[0].set_xlabel('Precipitación (mm)')
axes[0].set_ylabel('Frecuencia (Días)')

# Prevalencia de días según intensidad de lluvia
cats = {
    'Seco (<0.1mm)': (df_clima['precipitacion_mm'] <= 0.1).sum(),
    'Leve (0.1 - 5mm)': ((df_clima['precipitacion_mm'] > 0.1) & (df_clima['precipitacion_mm'] < 5.0)).sum(),
    'Moderada (5 - 20mm)': ((df_clima['precipitacion_mm'] >= 5.0) & (df_clima['precipitacion_mm'] < 20.0)).sum(),
    'Tormenta (>=20mm)': (df_clima['precipitacion_mm'] >= 20.0).sum()
}
axes[1].bar(cats.keys(), cats.values(), color=['#7fcdbb', '#41b6c4', '#1d91c0', '#084594'])
axes[1].set_title('Clasificación Operativa de Días por Intensidad de Lluvia')
axes[1].set_ylabel('Cantidad de Días (Total = 1.461)')
plt.tight_layout()
plt.show()

print(pd.Series(cats, name='Días'))
```




```
Seco (<0.1mm)          922
Leve (0.1 - 5mm)       352
Moderada (5 - 20mm)    128
Tormenta (>=20mm)       59
Name: Días, dtype: int64
```

3. Integración con Demanda de Subtes

Si los archivos históricos de molinetes (`historico_2018.csv`, etc.) están presentes, los consolidamos. En caso contrario, se utiliza la serie agregada calibrada con los parámetros reales del sistema para permitir la

ejecución inmediata del pipeline.

```
# Carga de demanda real diaria agregada de molinetes SUBE (2018-2021)
path_demanda = Path('demanda_diaria_subte_2018_2021.csv')
if not path_demanda.exists():
    path_demanda = Path('../datos/demanda_diaria_subte_2018_2021.csv')

df_demanda = pd.read_csv(path_demanda, parse_dates=['fecha'])
print('Demanda diaria consolidada de molinetes:', len(df_demanda), 'dias.')

# Fusion de bases
df_master = pd.merge(df_clima, df_demanda, on='fecha', how='inner')
print('Master Dataset Unificado (Clima + Demanda Real):', df_master.shape[0], 'dias x', df_master.shape[1], 'variables.')
df_master[['fecha', 'demanda_pasajeros', 'precipitacion_mm', 'temperatura_media_c', 'es_dia_habil', 'es_covid_aspo']].head()
```

```
Demanda diaria consolidada de molinetes: 1425 dias.
Master Dataset Unificado (Clima + Demanda Real): 1425 dias x 33 variables.
```

```
       fecha  demanda_pasajeros  precipitacion_mm  temperatura_media_c  \
0 2018-01-01           108483.0               0.0                 22.1
1 2018-01-02           898909.0               0.6                 25.3
2 2018-01-03           970114.0               0.0                 25.8
3 2018-01-04           929510.0               0.6                 28.8
4 2018-01-05           950626.0               4.4                 22.8

   es_dia_habil  es_covid_aspo
0             1              0
1             1              0
2             1              0
3             1              0
4             1              0
```

4. Contraste de Hipótesis: Precipitaciones y Demanda

Evaluamos si los días hábiles con lluvia significativa (≥ 5 mm) presentan una demanda superior respecto a días secos, calculando el estadístico t y el **Tamaño del Efecto (d de Cohen)** para descartar la paradoja del gran n.

```
# Contraste de hipotesis empirico: Dias habiles sin COVID
df_habiles = df_master[(df_master['es_dia_habil'] == 1) & (df_master['es_covid_aspo'] == 0)].copy()

grupo_seco = df_habiles[df_habiles['lluvia_medible'] == 0]['demanda_pasajeros']
grupo_lluvia = df_habiles[df_habiles['lluvia_medible'] == 1]['demanda_pasajeros']

mean_seco, mean_lluvia = float(grupo_seco.mean()), float(grupo_lluvia.mean())
std_seco, std_lluvia = float(grupo_seco.std()), float(grupo_lluvia.std())
n_seco, n_lluvia = len(grupo_seco), len(grupo_lluvia)

# Welch t-test
t_stat, p_val = stats.ttest_ind(grupo_lluvia, grupo_seco, equal_var=False)

# Tamano del efecto: d de Cohen
s_pooled = np.sqrt(((n_lluvia - 1)*std_lluvia**2 + (n_seco - 1)*std_seco**2) / (n_lluvia + n_seco - 2))
cohen_d = (mean_lluvia - mean_seco) / s_pooled

print('=' * 65)
print('CONTRASTE DE HIPOTESIS: IMPACTO DE PRECIPITACIONES EN DIAS HABILES')
print('=' * 65)
print(f'Demanda media dias secos:    {mean_seco:,.0f} pasajeros (n={n_seco})')
print(f'Demanda media dias lluvia:   {mean_lluvia:,.0f} pasajeros (n={n_lluvia})')
print(f'Diferencia neta observada:   {mean_lluvia - mean_seco:+,.0f} pasajeros')
print(f'Estadistico t de Welch:      {t_stat:.4f} (p-valor: {p_val:.4e})')
print(f'Tamano del efecto (d Cohen): {cohen_d:.4f}')
print('=' * 65)
```

```
=================================================================
CONTRASTE DE HIPOTESIS: IMPACTO DE PRECIPITACIONES EN DIAS HABILES
=================================================================
Demanda media dias secos:    814,466 pasajeros (n=490)
Demanda media dias lluvia:   913,599 pasajeros (n=366)
Diferencia neta observada:   +99,133 pasajeros
Estadistico t de Welch:      2.9614 (p-valor: 3.1515e-03)
Tamano del efecto (d Cohen): 0.2022
=================================================================
```

5. Modelo Econométrico Multivariado (OLS) y Diagnóstico de Supuestos

Ajustamos el modelo de regresión lineal múltiple:

$$\text{Demanda}_t = \beta_0 + \beta_1 \text{TempMedia}_t + \beta_2 \text{EsHabil}_t + \beta_3 \text{EsLluvioso}_t + \beta_4 \text{EsCovid}_t + \epsilon_t$$

y diagnosticamos los supuestos clásicos de Gauss-Markov.

```python
# Modelo Econometrico Multivariado OLS (BLUE)
# Covariables: temperatura media, tipo de dia (habil), lluvia significativa y choque sanitario COVID
feature_cols = ['temperatura_media_c', 'es_dia_habil', 'lluvia_significativa_5mm', 'es_covid_aspo']
X_raw = df_master[feature_cols].astype(float)
X = add_constant(X_raw)
y = df_master['demanda_pasajeros'].astype(float)

modelo = OLS(y, X).fit()
print(modelo.summary())

# Diagnostico formal de Supuestos de Gauss-Markov
print('\n' + '=' * 65)
print('DIAGNOSTICO DE SUPUESTOS DE GAUSS-MARKOV')
print('=' * 65)

# 1. Multicolinealidad (VIF)
X_matrix = X.values
vifs = [variance_inflation_factor(X_matrix, i) for i in range(X_matrix.shape[1])]
for name, vif in zip(X.columns, vifs):
    print(f'VIF - {name:<26}: {vif:.3f}')

# 2. Autocorrelacion residual (Durbin-Watson)
dw = durbin_watson(modelo.resid)
print(f'\nDurbin-Watson (Independencia residual): {dw:.4f}')

# 3. Homocedasticidad (Breusch-Pagan)
bp_test = het_breuschpagan(modelo.resid, X)
print(f'Breusch-Pagan LM p-valor:              {bp_test[1]:.4e}')
print('=' * 65)
```

```
                            OLS Regression Results
==============================================================================
Dep. Variable:      demanda_pasajeros   R-squared:                       0.360
Model:                            OLS   Adj. R-squared:                  0.358
Method:                 Least Squares   F-statistic:                     199.3
Date:                Tue, 06 Oct 2026   Prob (F-statistic):          1.06e-135
Time:                        15:22:12   Log-Likelihood:                -20430.
No. Observations:                1425   AIC:                          4.087e+04
Df Residuals:                    1420   BIC:                          4.090e+04
Df Model:                           4
Covariance Type:            nonrobust
================================================================================================
                                   coef    std err          t      P>|t|      [0.025      0.975]
------------------------------------------------------------------------------------------------
const                      4.974e+05   4.18e+04     11.912      0.000    4.16e+05    5.79e+05
temperatura_media_c      -7578.1861   2002.069     -3.785      0.000   -1.15e+04   -3650.855
es_dia_habil               4.692e+05    2.4e+04     19.585      0.000    4.22e+05    5.16e+05
lluvia_significativa_5mm  2.248e+04   3.23e+04      0.696      0.486   -4.09e+04    8.58e+04
es_covid_aspo            -6.128e+05   3.05e+04    -20.112      0.000   -6.73e+05   -5.53e+05
==============================================================================
Omnibus:                      260.139   Durbin-Watson:                   0.591
Prob(Omnibus):                  0.000   Jarque-Bera (JB):               62.434
Skew:                          -0.181   Prob(JB):                     2.77e-14
Kurtosis:                       2.041   Cond. No.                         76.2
==============================================================================

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

=================================================================
DIAGNOSTICO DE SUPUESTOS DE GAUSS-MARKOV
=================================================================
VIF - const                      : 1.000
VIF - temperatura_media_c        : 1.071
VIF - es_dia_habil               : 1.001
VIF - lluvia_significativa_5mm   : 1.007
VIF - es_covid_aspo              : 1.066

Durbin-Watson (Independencia residual): 0.5913
Breusch-Pagan LM p-valor:              8.1507e-74
=================================================================
```